



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського

Лабораторна робота №3
з дисципліни «Основи Проектування Трансляторів»

«РОЗРОБКА ГЕНЕРАТОРА КОДУ»

Виконав студент групи: КВ-33

Козлов Сергій

Мета роботи

Метою лабораторної роботи «Розробка генератора коду» є засвоєння теоретичного матеріалу та набуття практичного досвіду і практичних навичок розробки генераторів коду.

Постановка задачі

1. Розробити програму генератора коду (ГК) для підмножини мови програмування SIGNAL, заданої за варіантом.
2. Програма генератора коду має забезпечувати:
 - читання дерева розбору та таблиць, створених синтаксичним аналізатором, що було розроблено в лабораторній роботі №2;
 - виявлення семантичних помилок;
 - генерацію коду та/або побудову внутрішніх таблиць для генерації коду.
3. Зкомпонувати повний компілятор, що складається з розроблених раніше лексичного та синтаксичного аналізаторів і генератора коду, який забезпечує наступне:
 - генерацію коду та/або побудову внутрішніх таблиць для генерації коду;
 - формування лістингу вхідної програми з повідомленнями про лексичні, синтаксичні та семантичні помилки.

Граматика

Варіант 6

1. `<signal-program> --> <program>`
2. `<program> --> PROGRAM <procedure-identifier>;
<block>.`
3. `<block> --> <declarations> BEGIN <statements-list>
END`
4. `<statements-list> --> <empty>`
5. `<declarations> --> <constant-declarations>`
6. `<constant-declarations> --> CONST
<constant-declarations-list> | <empty>`
7. `<constant-declarations-list> -->
<constant-declaration> <constant-declarations-list> |
<empty>`
8. `<constant-declaration> --> <constant-identifier> =
<constant>;`
9. `<constant> --> '<complex-number>'`
10. `<complex-number> --> <left-part> <right-part>`
11. `<left-part> --> <unsigned-integer> | <empty>`
12. `<right-part> --> ,<unsigned-integer> | $EXP(
<unsigned-integer>) | <empty>`
13. `<constant-identifier> --> <identifier>`
14. `<procedure-identifier> --> <identifier>`
15. `<identifier> --> <letter><string>`
16. `<string> --> <letter><string> | <digit><string> |
<empty>`
17. `<unsigned-integer> --> <digit><digits-string>`
18. `<digits-string> --> <digit><digits-string> |
<empty>`
19. `<digit> --> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`
20. `<letter> --> A | B | C | D | ... | Z`

Архітектура генератора коду

Виконання семантичного аналізу і генерації коду напряду над CST (concrete syntax tree) ускладнене порівняно великою кількістю вузлів і глибиною такого дерева. Для спрощення роботи і для абстрагування від специфічних правил граматики мови SIGNAL було прийнято рішення ввести додатковий крок – згортку CST в AST (abstract syntax tree). Це значно спростить реалізацію семантичного аналізатора і генератора коду за рахунок меншої кількості типізованих вузлів та використання патерну Visitor (Відвідувач).

Генерація коду відбувається лише за відсутності лексичних, синтаксичних і семантичних помилок. Результатом роботи генератора є асемблерний код діалектом NASM під архітектуру x86-64 для ОС на базі ядра Linux. Однак доступні діалекти і архітектури можна розширити, додавши відвідувача з необхідною імплементацією.

Додатки

Додаток 1. Лістинг коду програми

Додаток 2. Результати тестування

Github

<https://github.com/Teollan/2026-opt-labs>